Travel Companion App

Week 8 | 04/03 - 04/09

Leader: Allen Paul

Varun Mange **Time Spent 14h**	Tasks completed: ● Updated our Github action to generate a test coverage artifact and use caching ● Fully deployed the backend with CDN for images ● Worked on the explore, destination details, and place details screens ● Added a /nearby endpoint on the backend that suggest places to visit close to the user's location ● Experiment with small AI chat models, compared their responses related to travel queries ● Tried to format their response as JSON Planned tasks for next week: ● Complete integrate the maps feature in our app ● Have an LLM running in our app that answers travel related queries Any issues or challenges: ●
Allen Paul **Time Spent 12h**	Tasks completed: ● Researched server-side caching and its role in improved backend performance. ● Explored different caching methods like in memory caching and API response caching ● Looked into potential technologies for implementation like Redis. ● Looked into parts of travel app where caching would reduce repeated API/database requests. ● Started working on frontend of OAuth . Planned tasks for next week: ● Any issues or challenges: ●
Darius Rafeh **Time Spent 10h**	Tasks completed: ● Map View Screen with MapsBox integration + Offline City Pack ● Offline City Pack uses local SQLite DB ● Map features like recentering when enabling location Planned tasks for next week: ● Setting up Mapbox API/Creating Account

	● Get location dropdown to work on my build in order to Demo Maps Screen Any issues or challenges: ● Pre-defined Location Select Dropdown when testing backend
Kapil Yadav **Time Spent 13h**	Tasks completed: ● Offline Pack Download: ○ I added a download button on the trip detail screen that pulls the destination's offline pack (places, phrases) from the API and saves it to AsyncStorage. It shows a downloaded badge with stats once complete so users know what's cached. ● Network Status + Offline Banner: ○ I set up a network listener using NetInfo that tracks connectivity app-wide and wired it into the root layout on boot. When you go offline, a red banner appears at the top of every screen letting you know some features are limited. ● Pack Version Check on Reconnect: ○ I built a hook that detects when the app comes back online and automatically checks if any downloaded offline packs have newer versions on the server. If an update is available it shows an alert so users know to re-download. ● Backend Service Tests: ○ I wrote test suites for the trip, user, and itinerary services covering CRUD, validation errors, duplicate username rejection, item ordering, and reorder logic. They follow the same pattern as the existing auth tests and clean up after themselves. ● Place Detail Screen: ○ I created a full place detail screen at app/place/[id] that shows rating, reviews, price level, address, phone, website, and curated/featured badges. Tapping any place card on the Explore tab now navigates directly to it. Planned tasks for next week: ● Any issues or challenges: ●

Total Time Spent: 49h

Summary: We finished deploying the backend. We finished working on the explore, destination, place details, and social connect screens. We implemented basic offline support by downloading places and phrases data. We expanded our test suite and covered more api endpoints. We started working on the maps screen.